

Handwriting OCR Grading System

(FastAPI + OpenAI Vision)

An end-to-end system for creating worksheet templates and automatically grading handwritten student submissions. The system provides four flows (templates) that can be used independently.

- Answer Boxes (rectangular answer fields)
- Dotted Lines (answers on dotted/guide lines)
- Blank Lines (underscore-style answer lines)
- Short Answer (answer boxes with reference answers)

The system uses OpenAI Vision through GPTOCRService for OCR extraction, with structured UI flows and REST APIs for template setup, preview, and grading.

Key Features

- Template setup UIs for four workflows
- OCR extraction via OpenAI Vision (GPTOCRService)
- Robust image utilities and detection services for each flow
- Standalone grading endpoints per flow
- Debug image support
- Organized template persistence under data/

Quick Start

1. Create virtual environment and install dependencies
 - Windows -
 - ☐ `python -m venv .venv`
 - ☐ `..venv\Scripts\activate`
 - Mac or Linux -
 - ☐ `python3 -m venv .venv`
 - ☐ `source .venv/bin/activate`
 - Install -
 - ☐ `pip install -r requirements.txt`
2. Configure environment variables
 - `OPENAI_API_KEY=your_openai_api_key`
 - `OPENAI_VISION_MODEL=gpt-4o`
3. Run the app
 - `uvicorn main:app --reload`
 - Visit `http://localhost:8000`

Project Structure

```
.
├── main.py                # Main FastAPI application entry point
├── main_crop.py           # Image cropping utility
├── main_ocr.py            # OCR processing script
├── app/
│   ├── __init__.py
│   ├── config.py          # Application configuration
│   └── services/          # Core service implementations
│       ├── __init__.py
│       ├── ai_enhancer.py  # AI-based answer enhancement
│       ├── answer_box_service.py # Answer box detection and grading
│       ├── blank_line_service.py # Blank line detection and grading
│       ├── dotted_line_service.py # Dotted line detection and grading
│       ├── gpt_ocr_service.py # OpenAI Vision OCR integration
│       ├── sa_service.py   # Short answer processing
│       └── text_post_processor.py # Text processing utilities
│   └── utils/              # Utility modules
│       ├── __init__.py
│       ├── border_detector.py # Border detection for worksheets
│       ├── config.py        # Application configuration settings
│       ├── config_utils.py  # Configuration utilities
│       ├── file_utils.py    # File handling utilities
│       ├── image_utils.py   # Image processing utilities
│       ├── logger.py        # Logging configuration and utilities
│       └── response_utils.py # API response formatting
├── data/                  # Template storage
│   ├── answer_templates/   # Answer box templates
│   ├── blank_templates/    # Blank line templates
│   ├── dotted_templates/   # Dotted line templates
│   └── sa_templates/       # Short answer templates
│       └── images/         # Template images
├── debug/                 # Debug output directory
├── models/                # ML models (if used)
├── static/                # Static assets (CSS, JS, images)
│   └── styles.css          # Main stylesheet
├── templates/             # HTML templates
│   ├── answer_box_creator.html # Answer box template creator
│   ├── blank_line_setup.html  # Blank line template setup
│   ├── dotted_line_setup.html # Dotted line template setup
│   ├── sa_setup.html         # Short answer template setup
│   └── unified_grader.html    # Unified grading interface
├── uploads/               # Temporary upload storage
├── .gitignore             # Git ignore file
├── README.md              # This file
└── requirements.txt        # Python dependencies
```

Architecture Overview

- Web framework: FastAPI
- Templating/UI: Jinja2 templates under templates/ and static assets under static/
- OCR service: app/services/**gpt_ocr_service.py** providing GPTOCRService
- Core detection and grading services:
 - ☐ Answer Box: AnswerBoxDetector, AnswerBoxGrader in **answer_box_service.py**
 - ☐ Dotted Line: DottedLineDetector, DottedLineTemplate in **dotted_line_service.py**
 - ☐ Blank Line: BlankLineDetector and grading logic in **blank_line_service.py**
 - ☐ Short Answer: ShortAnswerTemplateManager, ShortAnswerGrader in **sa_service.py**
- At app startup (main.py)
 - ☐ Services are initialized (e.g., AnswerBoxGrader(), DottedLineDetector(), BlankLineDetector())
 - ☐ Static and upload directories are mounted and ensured to exist.

OCR Service

- Resides in app/services/gpt_ocr_service.py
- In main.py, dependency injector get_ocr_service() provides GPTOCRService (api_key=OPENAI_API_KEY, model=gpt-4o)

Web UI Pages

Template Setup UIs

- /answer-box-creator - Create and manage answer box templates
- /dotted-line-setup - Set up worksheets with dotted line answer fields
- /blank-line-setup - Configure worksheets with blank (underscore) answer lines
- /sa-setup - Set up short answer templates with model answers

Grading Interface

- /unified_grader - Unified grading interface for all template types
- Supports grading worksheets with answer boxes, dotted lines, blank lines, or short answers
- Template selection and student submission in one place
- Displays results with visual feedback

How It Works

For Admins (Template Setup)

- Use the appropriate setup page for your worksheet type Upload a sample worksheet
- The system will automatically detect answer fields
- Enter correct answers and save the template
- Templates are stored in the data/ directory with unique IDs

For Grading

- Navigate to the unified grader
- Select the appropriate template Upload student submissions
- View automatic grading results with visual feedback

Template Management

All templates are stored in their respective directories under data/ with the following structure

- answer_templates/ - For answer box templates
- dotted_templates/ - For dotted line worksheets
- blank_templates/ - For blank line worksheets
- sa_templates/ - For short answer templates

Each template includes

- A JSON file with template metadata and answer keys
- Associated images (if any) in the images/ subdirectory

The setup UI includes an upload section where the admin can upload image files. The system automatically detects answer fields in the uploaded images. If any unwanted elements are detected, the admin can delete them. The admin can then enter the correct answers and save the template from the right-side panel. All templates are stored as JSON files in the data folder.

Admin Guide - Template Setup

This section is for administrators who need to create and manage worksheet templates.

Answer Box Templates

1. Navigate to Template Creator
 - Go to /answer-box-creator in your web browser
 - This interface allows you to create templates with rectangular answer boxes
2. Upload Worksheet
 - Click "Choose File" to upload a worksheet image
 - The system works best with high-contrast, well-defined boxes
 - Recommended box colors: orange or green for best detection
3. Configure Answer Boxes
 - Click "Detect Answer Boxes" to automatically find boxes
 - Review and adjust detected boxes as needed
 - Enter the correct answer for each box
4. Save Template
 - Click "Save Template" to store your configuration
 - The template will be available in the unified grader

Dotted Line Templates

1. Access Template Creator
 - Visit /dotted-line-setup in your browser
 - This is for worksheets with dotted or dashed answer lines
2. Upload and Configure
 - Upload your worksheet image
 - Click "Detect Dotted Lines" to identify answer areas Adjust any misidentified lines
 - Enter correct answers for each line
3. Save and Use
 - Save your template for use in the unified grader

Blank Line Templates

1. Open Template Creator
 - Go to /blank-line-setup
 - Designed for worksheets with blank (underscore) answer lines
2. Set Up Template
 - Upload your worksheet
 - Use "Detect Blank Lines" to identify answer areas
 - Review and adjust as needed Input correct answers
3. Save Configuration
 - Save the template for grading

Short Answer Templates

1. Start Template Creation
 - Navigate to /sa-setup
 - For worksheets with larger answer areas
2. Configure Answer Areas
 - Upload your worksheet
 - Click to detect answer boxes For each answer area:
 - Enter the model answer
 - Specify key words for evaluation
3. Complete Setup
 - Save your template

Tips for Best Results

- Use orange-colored boxes and lines for the best detection results.
- Add a green border to indicate the cropping area for students.
- When saving the answer field along with the correct answer, ensure that the uploaded image for detection is only within the green border.

User Guide - Unified Grader

The Unified Grader provides a single interface for grading all types of worksheets. Follow these steps to grade student submissions

1. Accessing the Grader

- Open your web browser and navigate to **/unified_grader**. You'll see a clean interface with template selection and upload options

2. Selecting a Template

- Choose the appropriate template type from the dropdown menu:
 - ☐ Answer Box
 - ☐ Dotted Line
 - ☐ Blank Line
 - ☐ Short Answer
- Select the specific template ID you want to use

3. Uploading Student Work

- Click "Choose File" to select a scanned or photographed student worksheet
- **Important:** Before uploading, ensure the image is cropped to include only the worksheet area with a green border around it
- The system uses the green border to properly align and grade the worksheet
- Make sure the image is clear and well-lit
- Click "Grade Worksheet" to process the submission and view results

4. Viewing Results

- The system will display the graded worksheet with visual indicators
- Correct answers are highlighted in green
- Incorrect answers are highlighted in red
- A summary of correct/incorrect answers is shown on the right side of the screen

Tips for Best Results

- Before uploading, ensure the image is cropped to include only the worksheet area with a green border around it (color printing not required)
- Ensure student work is clearly visible in the uploaded image
- For best OCR accuracy, make sure the worksheet is
 - ☐ Well-lit
 - ☐ Flat and aligned with the camera
 - ☐ Free from shadows and glares
- If results are unexpected, try retaking the photo with better lighting or alignment